Table of Contents

Using a Data Store **1**
Reading and Writing Asynchronously with Flow 1
Creating a Custom App / Launcher Icon for Kotlin Apps **7**
Creating the SVG 7
Importing the SVG 7
Setting the App Icon Label **8**
Creating a Splash Screen in Kotlin **10**
Adding the Dependency 11
Creating a Splash Screen Logo 11
Importing the Splash Screen 12
Setting the Splash Screen Theme 12

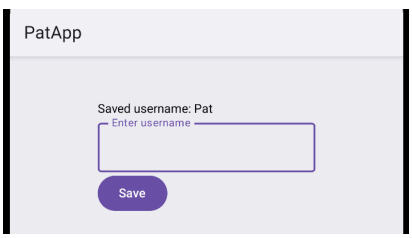
Using a Data Store

Kotlin's data store mechanism is useful for storing small amounts of data such as user preferences for an app. Using the data store avoids the complicated setup of a full database.

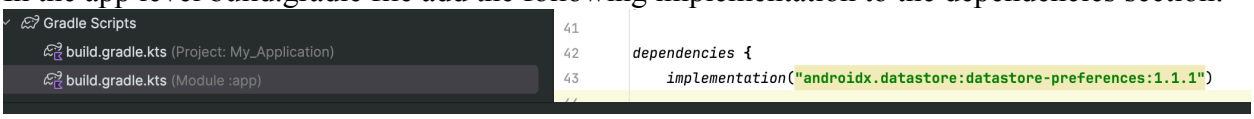
Example 1: Data Store

This example demonstrates how to create a data store to save a name (see Figure 1).

Figure 1: Enabling a Dynamic Data Store



In the app level build.gradle file add the following implementation to the dependencies section:




```
implementation("androidx.datastore:datastore-preferences:1.1.1")
```

Remember to click Sync Now to prompt Android Studio to add the dependency to the project.

Reading and Writing Asynchronously with Flow

The code in SettingsStore.kt stores data in key value pairs where unique keys are used to reference each value that is stored.

SettingsStore.kt implements uses Flow to provide asynchronous reads and writes. Kotlin recommends using Flow with data store so changes can be read reactively. To read something

reactively means you don't ask for the data every time. Instead, the data pushes updates to you whenever it changes.

Tip: Remember not to delete the project package name when this code is added to the Kotlin file in your project.

SettingsStore.kt

```
import android.content.Context
import androidx.datastore.preferences.core.*
import androidx.datastore.preferences.preferencesDataStore
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.flow.Flow

// Create DataStore attached to the Context
val Context.dataStore by preferencesDataStore("settings")

// Keys
private val USERNAME_KEY = stringPreferencesKey("username")

class SettingsRepository(private val context: Context) {

    // Read value (Flow)
    val usernameFlow: Flow<String> = context.dataStore.data.map { prefs ->
        prefs[USERNAME_KEY] ?: ""
    }

    // Save value (suspend)
    suspend fun saveUsername(name: String) {
        context.dataStore.edit { prefs ->
            prefs[USERNAME_KEY] = name
        }
    }
}
```

This code defines the screen that is displayed in Figure 1.

UserNameScreen.kt

```
@Composable
fun UsernameScreen(repo: SettingsRepository) {
    val scope = rememberCoroutineScope()

    // Observe stored username
    val username by repo.usernameFlow.collectAsState(initial = "")

    var input by remember { mutableStateOf("") }

    Column(modifier = Modifier.padding(100.dp)) {
        Spacer(modifier = Modifier.height(60.dp))
        Text("Saved username: $username")

        OutlinedTextField(
            value = input,
            onValueChange = { input = it },
            label = { Text("Enter username") }
        )

        Button(
            onClick = {
                scope.launch {
                    repo.saveUsername(input)
                    input = ""
                }
            }
        ) {
            Text("Save")
        }
    }
}
```

Finally, MainActivity.kt implements the DataStore interface by passing an instance of the repository to the composable.

MainActivity.kt

```
class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        val repo = SettingsRepository(this)
        setContent {
            UsernameScreen(repo)
        }
    }
}
```

Exercise 1 (1 mark)

How do we attach DataStore to the app Context?

using the preferencesDataStore delegate:

```
val Context.dataStore by preferencesDataStore("settings")
```

Exercise 2 (1 mark)

What is `stringPreferencesKey` and why is it used?

is a function that creates a unique key used to store and retrieve a String value in the DataStore. is used because DataStore stores data as key-value pairs, and key must be defined with the correct type so the DataStore knows what type of data to expect when reading or writing.

Exercise 3 (1 mark)

What other types of preference keys exist?

intPreferencesKey, booleanPreferencesKey, floatPreferencesKey, longPreferencesKey, doublePreferencesKey

Exercise 4 (1 mark)

What is Flow and why is it used?

Flow is used to automatically update the UI when data changes. Instead of manually checking for updates, Flow pushes the new data to the app whenever it changes.

Exercise 5 (1 mark)

Why is context passed into the SettingsRepository?

context is passed into SettingsRepository because DataStore is connected to the app's Context, and the repository needs it to access and use the DataStore.

Exercise 6 (1 mark)

Show how the *username* is declared in the composable and explain how it is assigned the data store value when the repository does not have a specific function to retrieve the value from storage.

```
val username by repo.usernameFlow.collectAsState(initial = “”)
```

It reads directly from the DataStore through usernameFlow. The collectAsState automatically updates the UI when the value changes, and initial = "" sets the starting value to empty.

Exercise 7 (1 mark)

What UI behavior would you expect when usernameFlow changes?

UI automatically updates to display new username without needing to manually refresh the screen.

Exercise 8 (1 mark)

What advantage does the data store have over a Room database?

DataStore is simpler to set up than Room and requires less code, also is ideal for storing small amounts of data like user preferences, without needing tables.

Exercise 9 (4 marks)

Modify Example 1 to enable the storage of both a first name and last name. Each name must be stored separately. Show your revised SettingsStore.kt and UserNameScreen.kt file here:

```
package com.example.myapplication

import android.content.Context
import androidx.datastore.preferences.core.*
import androidx.datastore.preferences.preferencesDataStore
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.flow.Flow

// Create DataStore attached to the Context
val Context.dataStore by preferencesDataStore("settings")

// Keys
private val USERNAME_KEY = stringPreferencesKey("username")
private val LASTNAME_KEY = stringPreferencesKey("lastname")


class SettingsRepository(private val context: Context) {

    // Read value (Flow)
    val usernameFlow: Flow<String> = context.dataStore.data.map { prefs ->
        prefs[USERNAME_KEY] ?: ""
    }
    val lastnameFlow: Flow<String> = context.dataStore.data.map { prefs ->
        prefs[LASTNAME_KEY] ?: ""
```

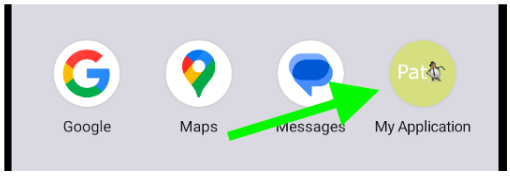
```
package com.example.myapplication

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import kotlinx.coroutines.launch
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue

@Composable
fun UsernameScreen(repo: SettingsRepository) {
```

Creating a Custom App / Launcher Icon for Kotlin Apps

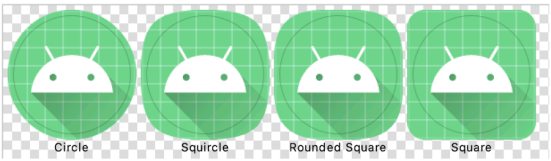
Example 2: Creating an App Icon

This next section will discuss how to create an app icon for the mobile home screen like the one below. (When you complete this example, create an icon with your name and an animal that starts with the first letter of your name).

Figure 2: Viewing the App (Launcher) Icon



Depending on the manufacturer, the home screen icon can take on varying shapes from rounded to squared.



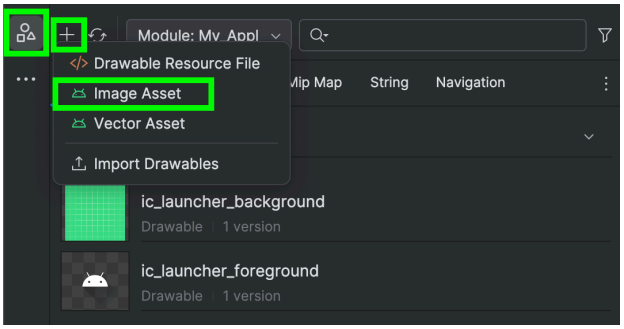
Fortunately, Kotlin can use adaptive icons that are created in SVG format to manage the different size, shape and densities for us on new and old devices.

Creating the SVG

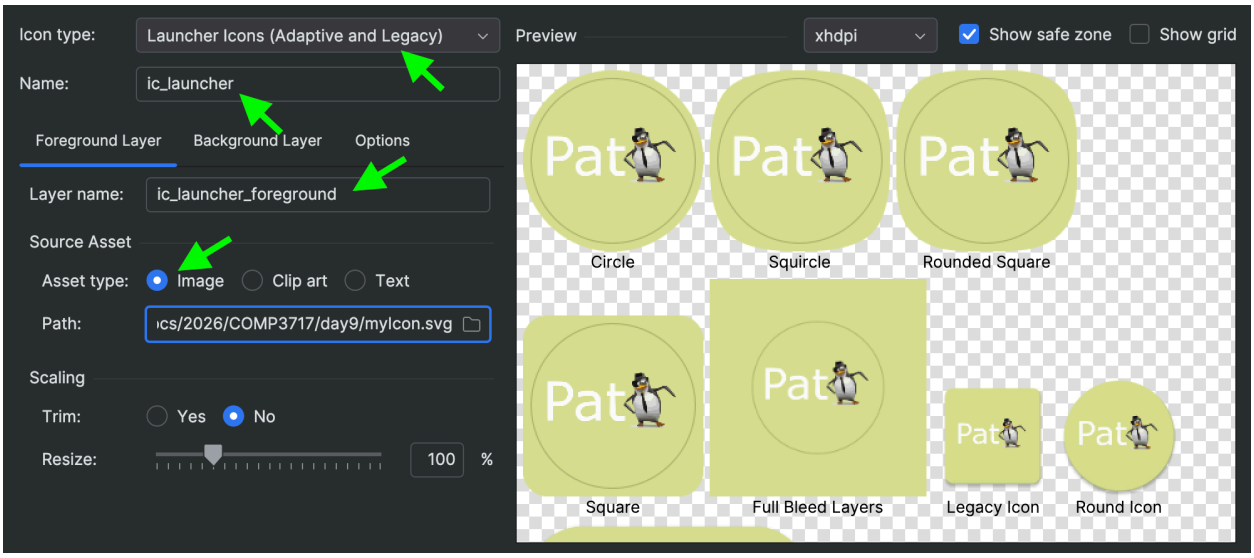
One way to create a suitable SVG involves making a 432x432 pixel image in *.png format. The image can then be converted to SVG at https://vectorise.me/.

Importing the SVG

After downloading the *.svg, from the Android Studio resource manager choose to add an image asset to import the *.svg into the project.



Keep the settings shown below and make sure the name is **ic_launcher** and that the Layer name is set to **ic_launcher_foreground**.



Select all defaults to complete the app icon import into the appropriate mipmap folders within the **res** directory.

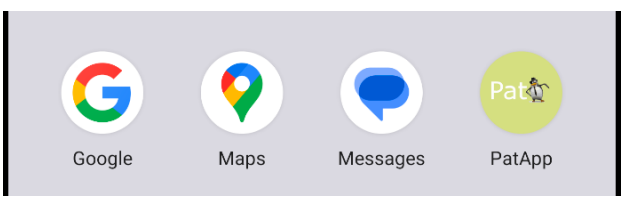
After building and running the app, an app icon like the one in Figure 2 will appear on the home screen.

Setting the App Icon Label

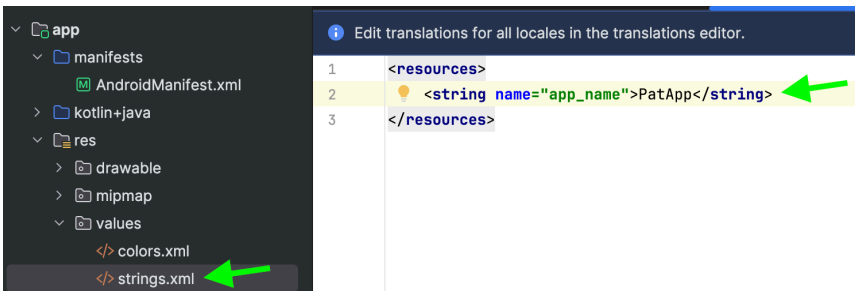
Example 3: Setting the App Icon Label

This example shows how to set the app icon label. For my case, I labeled it *PatApp*.

Figure 3: Custom App Icon with Custom Name



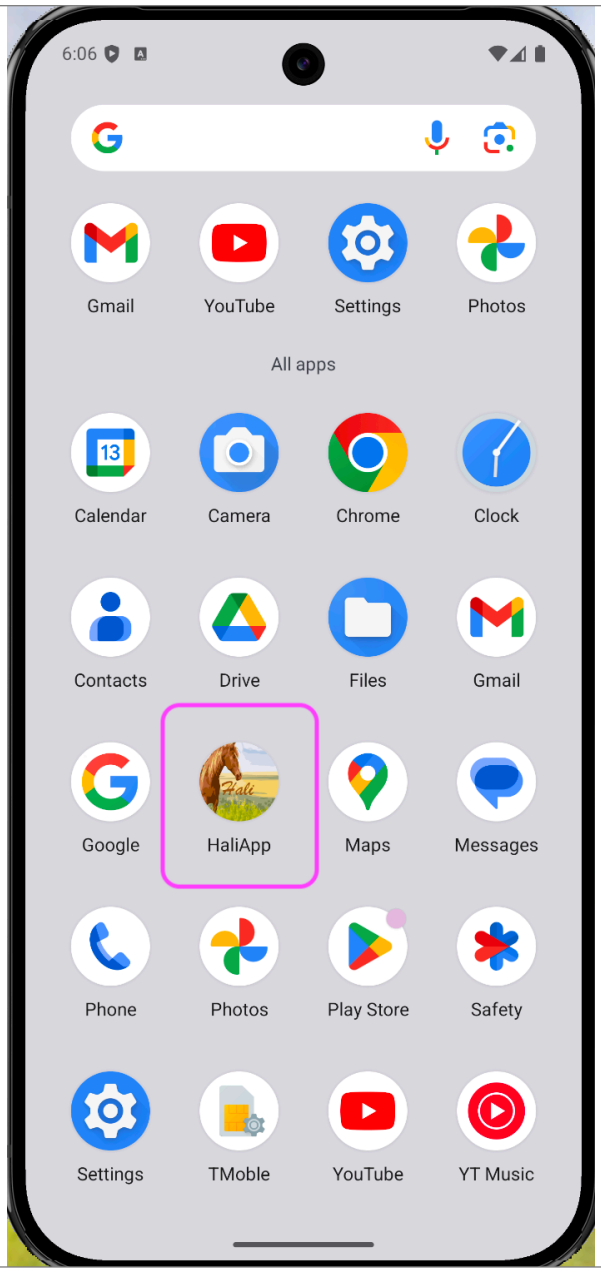
The app name that appears on the Android home screen is set by adjusting **app_name** element in the **strings.xml** file.



After building and running your project, a suitably named app icon will now be visible on the home screen like the one in Figure 3.

Exercise 10 (4 marks)

Set the app name to your name. In the icon, include an image of an animal that has the same first letter as your name. Show a screenshot of your app icon on the homescreen of the emulator here:



Creating a Splash Screen in Kotlin

In modern Android Kotlin apps, the officially recommended way to create a splash screen is to use the Android 12+ SplashScreen API. SplashScreen API works in current and old phones back to Android 5.0+ (2014).

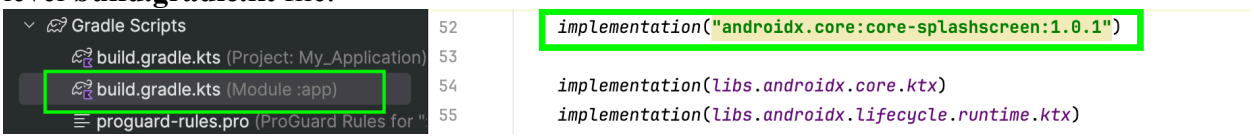
Example 4: Creating a Splash Screen

This example describes how to create a splash screen with a grey background. (When you do this create a logo with your name and an animal that starts with the first letter of your name).

Figure 4: Viewing the splash screen during an app launch

Adding the Dependency

To enable the splash screen, add this implementation within the **dependencies** section of the app level **build.gradle.kt** file:




```
implementation("androidx.core:core-splashscreen:1.0.1")
```

Remember to click **Sync Now** after adding the dependency implementation.

Creating a Splash Screen Logo

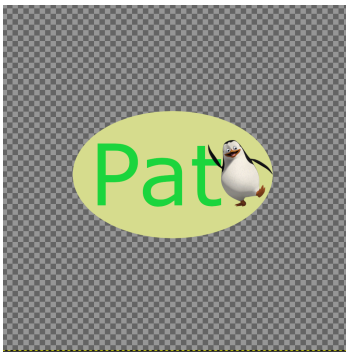
For splash screens, Google recommends creating a logo with a transparent background. Table 1 shows guidelines for creating a splash screen logo.

Table 1: Splash Screen Guidelines

Setting	Recommendation
Format	SVG's are recommended because they scale cleanly across different screens with different densities.
Canvas size	512×512 or 1024×1024 (1024 recommended for clarity)
Safe area	Keep main icon inside 70–80% of canvas
Padding	~10–15% around edges
No background	Transparent background (Android sets fill color)

Given the guidelines in Table 1, I recommend creating a 1024x1024 logo with appropriate safe area, padding and no background. The image could be like the one in Figure 5 but with your name instead and an animal that has the same first letter as your name.

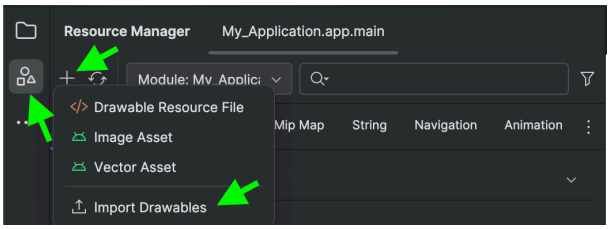
Figure 5: Logo with no background



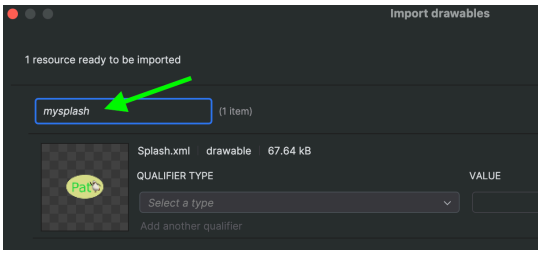
If you save the file as a png it can be converted to svg at https://vectorise.me/ .

Importing the Splash Screen

From the resource manager, import the splash screen as a drawable.

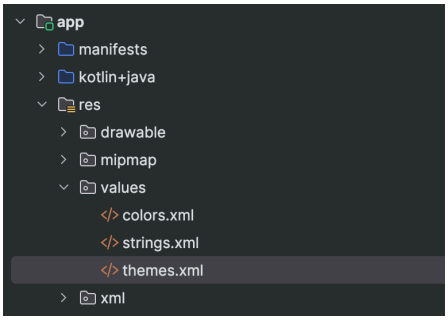


Be sure to name the splash screen as **mysplash**.



Setting the Splash Screen Theme

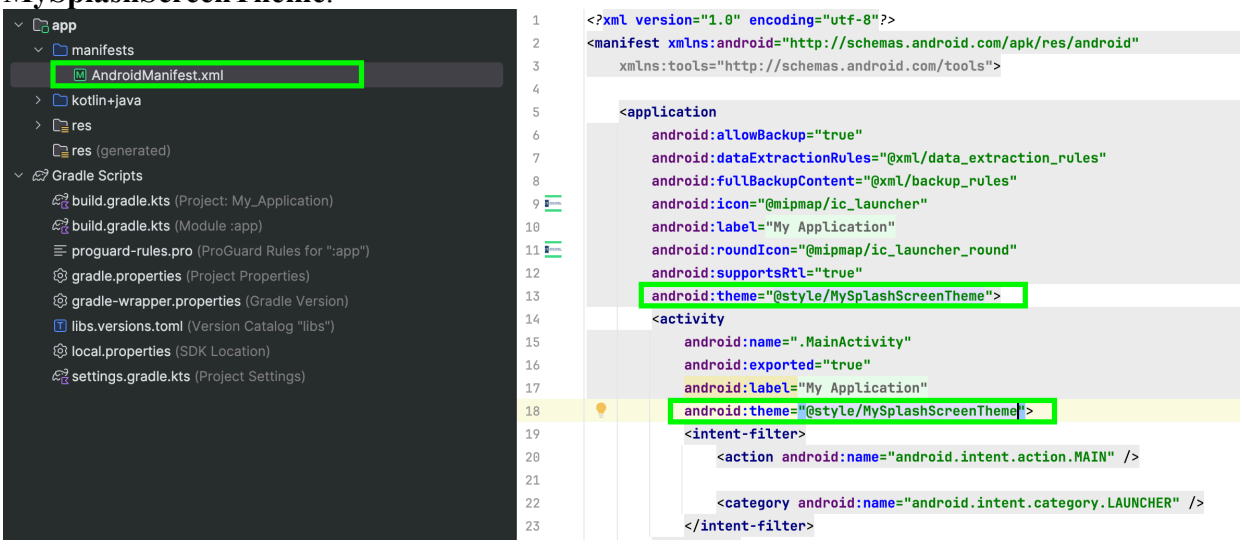
The splash screen theme can set the splash screen drawable reference and background color. To do this, navigate to **res | values | themes.xml**.



In themes.xml, add this new style element with the name attribute of **MySplashScreenTheme**. The application theme actually needs to be the name of your application since it was generated when your app project was created. The application theme name should already be present in an existing style tag like the second style element below.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="MySplashScreenTheme" parent="Theme.SplashScreen">
 <item name="windowSplashScreenAnimatedIcon">@drawable/mysplash</item>
        <item name="windowSplashScreenBackground">#999999</item>
        <item name="postSplashScreenTheme">@style/Theme.MyApplication</item>
        <item name="windowSplashScreenIconBackgroundColor">#000000</item>=
    </style>

    <style name="Theme.MyApplication"
parent="android:Theme.Material.Light.NoActionBar" />
</resources>
```

After creating the theme, modify the AndroidManifest.xml file so it references the **MySplashScreenTheme**.



Next, create the SplashViewModel class in **SplashViewModel.kt** to launch the splash screen. For exaggeration, this file displays the splash screen for 10 seconds so you likely will want to shorten this duration before deploying the app.

```
class SplashViewModel : ViewModel() {
    private val mutableStateFlow = MutableStateFlow(true)
    val isLoading = mutableStateFlow.asStateFlow()

    init {
        viewModelScope.launch {
            delay(10000)
            mutableStateFlow.value = false
        }
    }
}
```

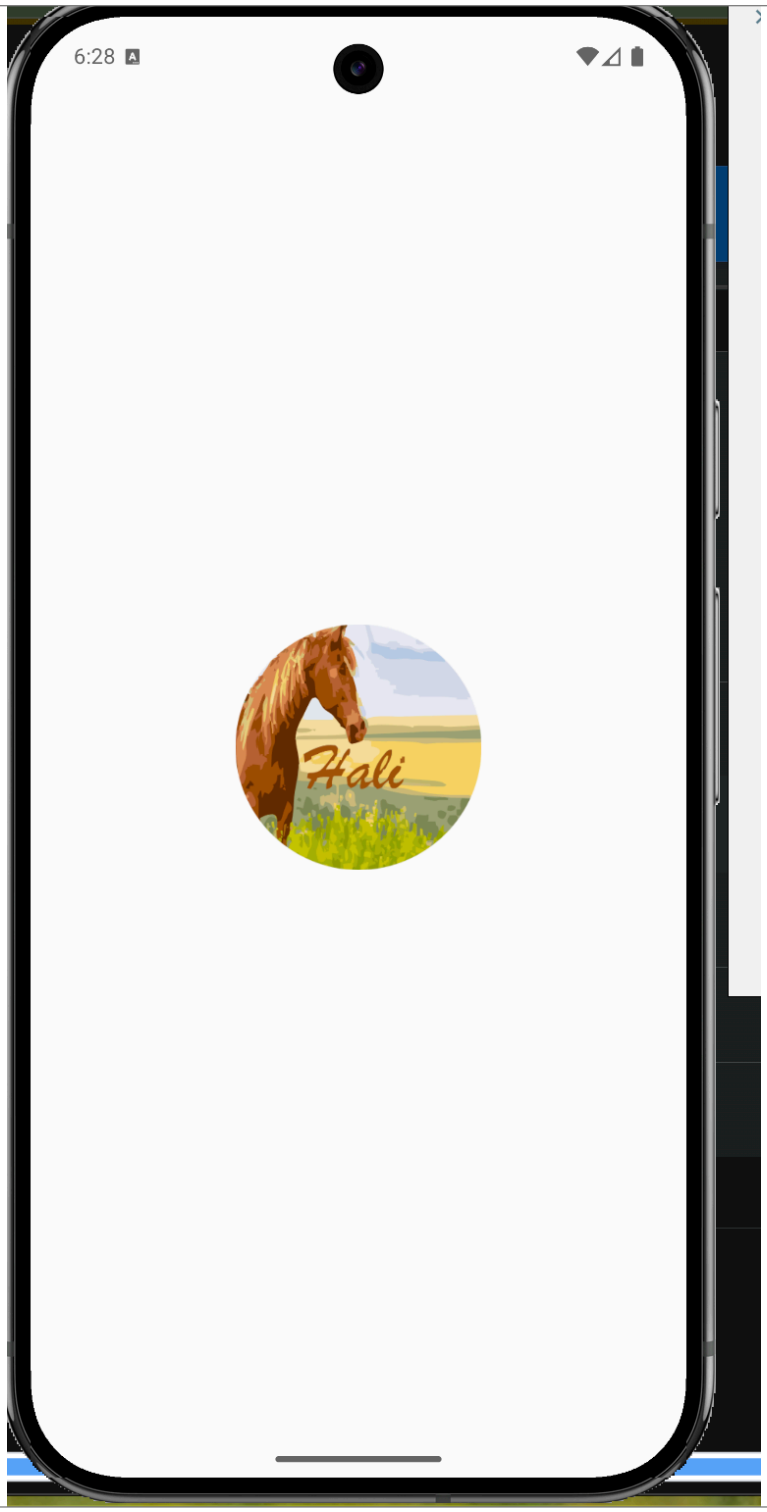
Finally, modify **MainActivity.kt** to launch the splash screen. Instructions which enable the custom splash screen are highlighted.

```
class MainActivity : ComponentActivity() {
    private val viewModel: SplashViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        val splashScreen = installSplashScreen()
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        splashScreen.setKeepOnScreenCondition{viewModel.isLoading.value}
        setContent {
            MyApplicationTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(
    name: String,
    modifier: Modifier = Modifier
) {
    Text(
        text = "Hello $name!",
        style = MaterialTheme.typography.headlineMedium,
        modifier = modifier
    )
}
```

At this point, your splash screen should launch when you restart the app.

Exercise 11 (4 marks)

Show a screenshot of the launch screen while it is running in your emulator. The splash screen should show your name and an animal that shares the same first letter as your name.


6:28
Hali